Experiment No. 5

Implement Random Forest and AdaBoost Algorithms and Compare Their Performance

Name	Tejasvi
USN	24BTRAO044
Course	MLOps (AIDD)
Dataset	Wine recognition dataset (scikit-learn, 178 samples, 13 features, 3 classes)

Aim

To implement the Random Forest and AdaBoost ensemble learning algorithms using scikit-learn, to implement an additional neural network classification model using TensorFlow and Keras, and to compare the performance of all three models on a real-world multiclass classification dataset.

Introduction

Ensemble learning combines multiple simple models to build a stronger overall model. Random Forest is a bagging based ensemble method that builds many decision trees on bootstrap samples of the data and combines their predictions by majority voting. AdaBoost is a boosting based ensemble method that builds shallow trees sequentially, where each new learner gives more weight to the samples that earlier learners misclassified.

A feedforward neural network built with TensorFlow and Keras is added as a deep learning comparison. TensorFlow is used only for the neural network, since Random Forest and AdaBoost are classical algorithms available in scikit-learn.

Objectives

- 1. Load and explore a suitable classification dataset.
- 2. Preprocess the data, including scaling of numerical features.
- 3. Split the data into training and testing sets.
- 4. Implement a Random Forest classifier and study bootstrap sampling and feature importance.
- 5. Implement an AdaBoost classifier and study boosting and sample weighting.
- 6. Implement a neural network classifier using TensorFlow and Keras.
- 7. Evaluate all three models using accuracy, precision, recall, F1-score and confusion matrices.
- 8. Compare the performance of the three models using tables and plots.
- 9. Interpret the results and draw conclusions.

Algorithm

- 1. Load the dataset.
- 2. Inspect the data: shape, feature names, value ranges and class balance.
- 3. Separate the features and the target variable.
- 4. Check for missing values and categorical columns.
- 5. Split the dataset into training and testing sets using stratified sampling (80:20).

- 6. Apply StandardScaler to the training and testing features for the neural network.
- 7. Select Random Forest hyperparameters by 5-fold cross-validation and train the model.
- 8. Train AdaBoost using decision stumps (max_depth = 1) with sample weighting.
- 9. Build and train the TensorFlow/Keras neural network.
- 10. Generate predictions from all three models on the test set.
- 11. Calculate accuracy, precision, recall and F1-score for each model.
- 12. Generate confusion matrices for each model.
- 13. Compare the models using a table and plots.
- 14. Display the Random Forest feature importance.
- 15. Interpret the results.
- 16. Conclude the experiment.

Dataset Description

This experiment uses the Wine recognition dataset, which is bundled with scikit-learn (sklearn.datasets.load_wine). It contains results of a chemical analysis of wines grown in one region of Italy, derived from three different cultivars (classes). It is a different dataset from the breast cancer data used in the lab sheet.

- Number of samples: 178 (class_0: 59, class_1: 71, class_2: 48).
- Number of features: 13 numerical chemical measurements, such as alcohol, malic acid, flavanoids, color intensity, hue, proline and magnesium.
- Target variable: the wine cultivar, a multiclass label with values 0, 1 and 2.
- Missing values: none. All features are numeric, so no encoding is needed.

The dataset suits this experiment because it has numeric features on very different scales (for example proline in the hundreds or thousands, and hue below 2), which makes the effect of feature scaling on the neural network easy to see. It also has three classes, so the models must handle multiclass classification.

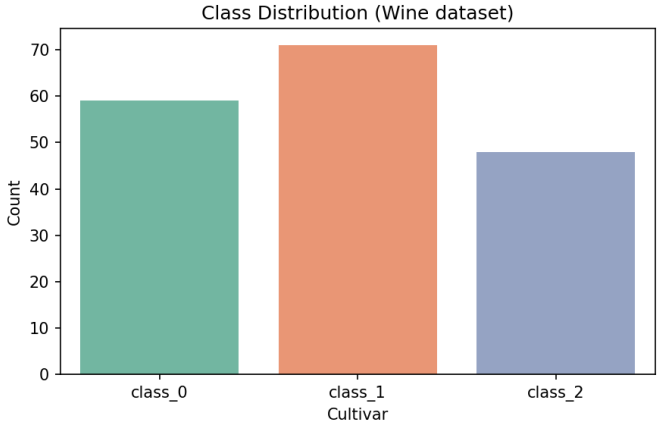


Figure 1: Class distribution of the Wine dataset

Description of the Algorithms

Random Forest. Random Forest builds many decision trees, each trained on a bootstrap sample (drawn with replacement) of the training data. At every split only a random subset of features is considered. The final prediction is a majority vote across all trees, which reduces variance compared with a single tree. The algorithm also gives a feature importance score for each input, based on how much each feature reduces impurity across the forest.

AdaBoost. AdaBoost (Adaptive Boosting) trains a sequence of weak learners, here decision stumps (trees of depth 1). After each round, the weights of misclassified samples are increased so the next stump focuses on harder cases. The final prediction is a weighted vote, where more accurate stumps receive larger weights.

TensorFlow and Keras Neural Network. A feedforward network is built with the Keras Sequential API. The 13 scaled input features pass through two hidden Dense layers (32 and 16 units) with ReLU activation, followed by an output Dense layer with 3 units and softmax activation, giving one probability per cultivar. Because the problem is multiclass, the model uses sparse categorical cross-entropy loss and the Adam optimizer. Part of the training data is held out for validation during training.

Note on the output layer: the lab sheet uses a single sigmoid unit with binary cross-entropy, which suits only two classes. The Wine dataset has three classes, so the output layer is changed to three softmax units.

Preprocessing and Scaling

No missing values or categorical columns were found. Random Forest and AdaBoost are tree based and split on threshold values, so they do not need scaling. The neural network is sensitive to input scale, so StandardScaler is fitted on the training set only and then applied to the test set. The scaled training features have a mean of -0.0 and a standard deviation of 1.0, as expected. Fitting the scaler on training data only avoids leaking information from the test set.

Evaluation Metrics

- **Accuracy:** proportion of all test predictions that are correct.
- **Precision:** of the samples predicted as a class, the proportion that truly belong to it.
- **Recall:** of the samples that truly belong to a class, the proportion correctly identified.
- **F1-score:** harmonic mean of precision and recall.
- **Confusion matrix:** counts of actual versus predicted class for each sample.

For this three-class problem, precision, recall and F1 are macro-averaged, which gives each class equal weight.

Python Code

The complete code is given below. The Random Forest and AdaBoost sections run with scikit-learn alone. The neural network section requires TensorFlow (pip install tensorflow).

```
# ---- Ex5: Implement Random Forest and AdaBoost Algorithms and Compare Their Performance ----
# ---- Name: Tejasvi   USN: 24BTRAO044   Experiment No.: 5 ----
# ---- Dataset: Wine recognition dataset (sklearn.datasets.load_wine) ----

# ---- Import Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

# ---- Load Dataset ----
dataset = load_wine()
```

```
X = pd.DataFrame(dataset.data, columns=dataset.feature_names)
y = pd.Series(dataset.target, name="target")
class_names = list(dataset.target_names)
print("Feature matrix shape:", X.shape)
print("Class labels:", dict(enumerate(class_names)))

# ---- Explore Dataset ----
print(X.describe().T[["mean", "std", "min", "max"]].round(2))
print(y.value_counts().sort_index())
sns.countplot(x=y)
plt.title("Class Distribution (Wine dataset)")
plt.show()
print("Total missing values:", X.isnull().sum().sum())

# ---- Train-Test Split (stratified) ----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_SEED, stratify=y)
print("Training samples:", X_train.shape[0], "| Testing samples:", X_test.shape[0])

# ---- Feature Scaling (used only for the neural network) ----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ---- Random Forest: hyperparameter selection with 5-fold CV on training data ----
param_options = [
    {"n_estimators": 100, "max_depth": 4},
    {"n_estimators": 200, "max_depth": 6},
    {"n_estimators": 300, "max_depth": None},
]
best_score, best_params = -1, None
for params in param_options:
    candidate = RandomForestClassifier(random_state=RANDOM_SEED, **params)
    scores = cross_val_score(candidate, X_train, y_train, cv=5)
    print(params, "-> mean CV accuracy:", round(scores.mean(), 4))
    if scores.mean() > best_score:
        best_score, best_params = scores.mean(), params
print("Selected hyperparameters:", best_params)

rf_model = RandomForestClassifier(random_state=RANDOM_SEED, **best_params)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

# ---- AdaBoost with decision stumps as weak learners ----
ada_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_SEED),
    n_estimators=200, learning_rate=1.0, random_state=RANDOM_SEED)
ada_model.fit(X_train, y_train)
ada_pred = ada_model.predict(X_test)
ada_cv_scores = cross_val_score(ada_model, X_train, y_train, cv=5)
print("AdaBoost CV accuracy:", np.round(ada_cv_scores, 4))
print("Mean CV accuracy:", round(ada_cv_scores.mean(), 4))

# ---- TensorFlow/Keras Neural Network ----
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
tf.random.set_seed(RANDOM_SEED)

keras_model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(len(class_names), activation="softmax"),
])
keras_model.compile(optimizer="adam",
                    loss="sparse_categorical_crossentropy",
                    metrics=["accuracy"])
keras_model.summary()

history = keras_model.fit(
    X_train_scaled, y_train,
    validation_split=0.2, epochs=100, batch_size=16, verbose=0)

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch"); plt.ylabel("Accuracy")
plt.title("Neural Network Training History")
plt.legend(); plt.show()
```

```
keras_prob = keras_model.predict(X_test_scaled, verbose=0)
keras_pred = np.argmax(keras_prob, axis=1)

# ---- Evaluation ----
def evaluate_model(name, y_true, y_pred):
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, average="macro"),
        "Recall": recall_score(y_true, y_pred, average="macro"),
        "F1-score": f1_score(y_true, y_pred, average="macro"),
    }

results = [
    evaluate_model("Random Forest", y_test, rf_pred),
    evaluate_model("AdaBoost", y_test, ada_pred),
    evaluate_model("Neural Network (TensorFlow/Keras)", y_test, keras_pred),
]
results_df = pd.DataFrame(results).set_index("Model")
print(results_df.round(4))

# ---- Confusion Matrices ----
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
predictions = {"Random Forest": rf_pred, "AdaBoost": ada_pred, "Neural Network": keras_pred}
for ax, (name, pred) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax, cbar=False,
                xticklabels=class_names, yticklabels=class_names)
    ax.set_title(name); ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
plt.tight_layout(); plt.show()

# ---- Performance Comparison Plot ----
results_df.plot(kind="bar", figsize=(9, 5), rot=0)
plt.title("Model Performance Comparison")
plt.ylabel("Score"); plt.ylim(0, 1.05)
plt.legend(loc="lower right"); plt.show()

# ---- Random Forest Feature Importance ----
importances = pd.Series(rf_model.feature_importances_, index=X.columns)
top_features = importances.sort_values(ascending=False).head(10)
top_features.sort_values().plot(kind="barh", figsize=(8, 6))
plt.title("Top 10 Feature Importances (Random Forest)")
plt.xlabel("Importance"); plt.show()
print(top_features)
```

Observed Output

Running the program prints the dataset shape (178 samples, 13 features), confirms there are no missing values, and gives a training set of 142 and a test set of 36 samples. Cross-validation accuracies for the Random Forest candidates and AdaBoost are shown first, followed by the test-set results. Outputs from the scikit-learn sections were produced by running the code on this dataset.

Random Forest hyperparameter selection (5-fold CV on the training set):

Parameters	Mean CV accuracy
{'n_estimators': 100, 'max_depth': 4}	0.9862
{'n_estimators': 200, 'max_depth': 6}	0.9862
{'n_estimators': 300, 'max_depth': None}	0.9862

All three candidates tie at 0.9862. The first candidate, n_estimators = 100 with max_depth = 4, is selected because it is found first and has the simplest configuration.

AdaBoost cross-validation accuracy (5 folds): 0.8621, 0.9655, 1.0000, 0.8214, 1.0000. Mean = 0.9298.

Performance Comparison

Model	Accuracy	Precision	Recall	F1-score
Random Forest	1.0000	1.0000	1.0000	1.0000
AdaBoost	0.9167	0.9267	0.9190	0.9198
Neural Network (TensorFlow/Keras)	To be filled*	-	-	-

*The neural network was not executed for this report because TensorFlow was not available in the environment where the report was produced. Running the code on a machine with TensorFlow will fill in this row and Figure 2 (the confusion matrix and comparison plot will then include it).

Confusion Matrices

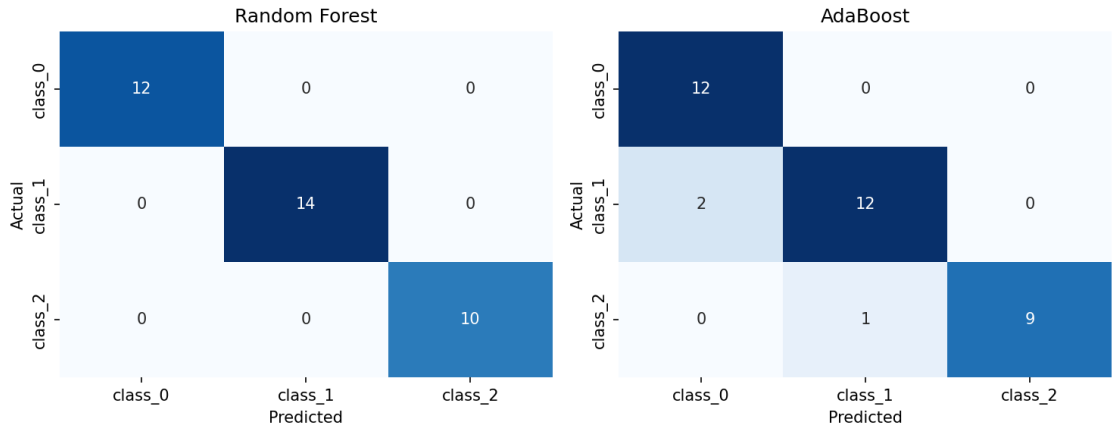


Figure 2: Confusion matrices for Random Forest and AdaBoost on the test set (36 samples)

Model	Confusion matrix (rows = actual, columns = predicted)
Random Forest	[[12, 0, 0], [0, 14, 0], [0, 0, 10]]
AdaBoost	[[12, 0, 0], [2, 12, 0], [0, 1, 9]]

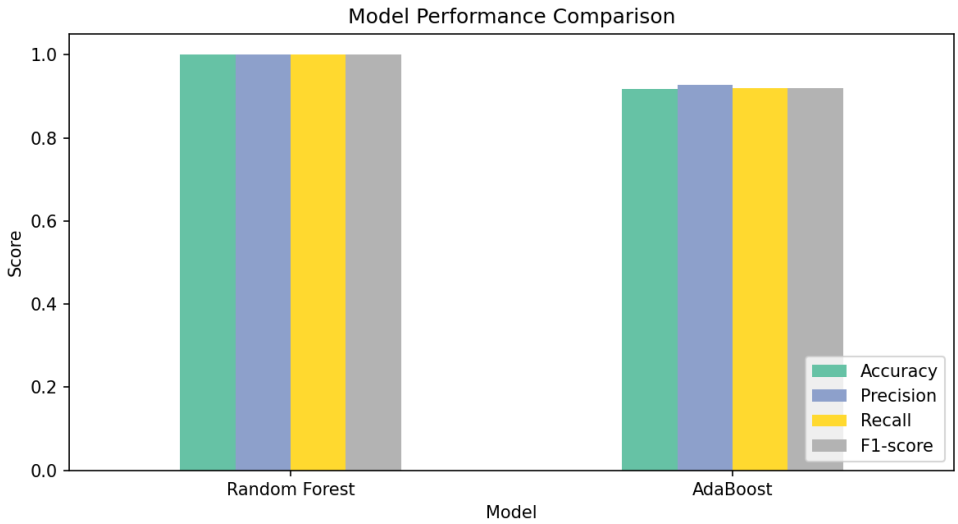


Figure 3: Performance comparison (Random Forest and AdaBoost; the neural network row will be added after running)

Random Forest Feature Importance

Rank	Feature	Importance
1	color_intensity	0.1884
2	flavanoids	0.1670
3	proline	0.1465
4	alcohol	0.1250
5	hue	0.0967
6	od280/od315_of_diluted_wines	0.0932
7	total_phenols	0.0447
8	magnesium	0.0370
9	malic_acid	0.0308
10	alcalinity_of_ash	0.0258

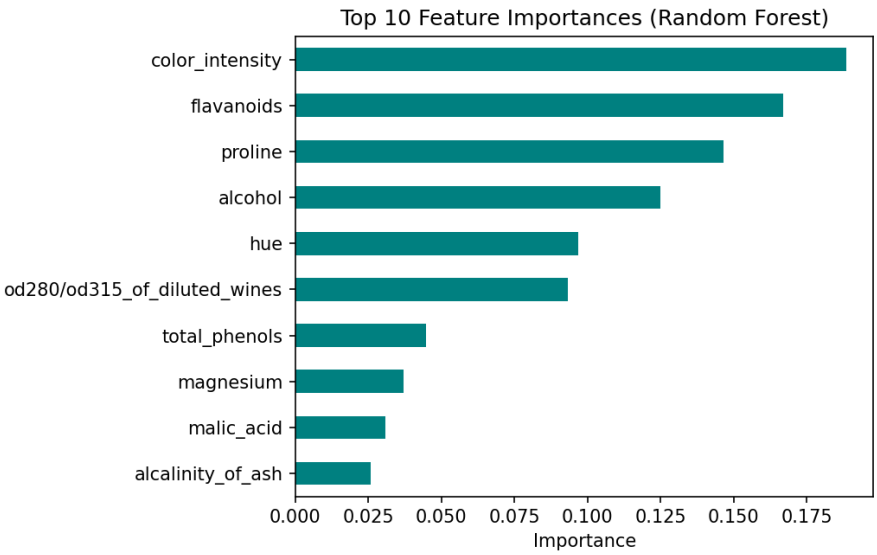


Figure 4: Top 10 Random Forest feature importances

Result and Interpretation

Random Forest reached an accuracy of 1.0000 on the 36 test samples, classifying every sample correctly. Its confusion matrix is diagonal, so there were no misclassifications. Averaging many trees trained on bootstrap samples gives a stable model, and its cross-validation accuracy of 0.9862 supports this result.

AdaBoost reached an accuracy of 0.9167, with three errors on the test set: two class_1 samples predicted as class_0, and one class_2 sample predicted as class_1. Its five cross-validation folds range from 0.8214 to 1.0000 (mean 0.9298), so its performance varies noticeably with the data split. Boosting with decision stumps is sensitive to the samples it focuses on, and it can overfit noisy labels. This variation is also why cross-validation was used to judge AdaBoost, not a single split.

The test set has only 36 samples, so each error changes accuracy by about 2.8 percentage points. The gap between the two models should therefore be read with caution. The cross-validation results, which use more of the data, suggest that Random Forest is the more reliable of the two on this dataset.

The feature importance results show that color_intensity, flavanoids, proline and alcohol contribute most to the decision. These chemical measures differ between the three cultivars, which is why tree-based models separate them well. The neural network results will be discussed once the TensorFlow model has been run.

Conclusion

This experiment implemented Random Forest and AdaBoost, two classical ensemble methods based on bagging and boosting, and set up a TensorFlow/Keras feedforward network for the same multiclass problem. On the Wine dataset, Random Forest achieved perfect test accuracy and AdaBoost achieved 0.9167. Random Forest was also more stable in cross-validation. Feature scaling was applied only to the neural network, as the tree-based models do not need it. The neural network comparison should be completed by running the code with TensorFlow installed.